

INFORME TÉCNICO DE VALIDACIÓN

Aislamiento de memoria por usuario y personalidad

Núcleo AI-first · Fábrica multi-app (Sina, Aurora, ... 36+ apps)

Campo	Valor
Proyecto	Núcleo AI-first (Orquestador de IA)
Repositorio	https://github.com/hanccotemp/kernel_app
Documento	Validación de memoria aislada por (personalidad + usuario)
Versión	1.0
Fecha	2026-06-01
Estado	APROBADO (nivel "Ideal", con nota de persistencia)
Reproducible con	node scripts/test-memoria.js

Contenido

1. Resumen ejecutivo

Este informe valida, con evidencia ejecutable, que el núcleo mantiene la memoria de conversación SEPARADA por cada usuario dentro de cada personalidad de IA. La llave de aislamiento es la combinación (personalidad/app + usuario), no solo el usuario ni solo la personalidad.

Veredicto: APROBADO. Las cuatro pruebas exigidas (A–D) pasan, más una prueba adicional de borrado LGPD (E). El único elemento pendiente es la persistencia física en base de datos (PostgreSQL); la llave de aislamiento ya es la correcta, por lo que corresponde al nivel “Aceptable con nota” previsto en el criterio del solicitante.

Prueba	Qué valida	Resultado
A	Aislamiento entre usuarios de la misma personalidad	✓ Pasa
B	Continuidad de memoria del mismo usuario	✓ Pasa
C	Aislamiento entre personalidades	✓ Pasa
D	Escala (12 usuarios en una personalidad)	✓ Pasa (12/12)
E	Borrado LGPD (derecho a eliminar)	✓ Pasa

2. Objetivo y alcance

Confirmar técnicamente que un único Orquestador de IA, que sirve a varias personalidades (Astrólogo, Religioso, Letrado, Psicólogo, ...), mantiene para cada usuario una memoria propia y totalmente aislada dentro de cada personalidad.

Dentro de alcance: modelo de almacenamiento de memoria, llave de aislamiento, mecánica de cada caso de prueba, evidencia de ejecución, y la ruta a persistencia en PostgreSQL.

Fuera de alcance: rendimiento bajo carga real de producción y la implementación física de PostgreSQL (planificada, no incluida en esta validación).

3. Requisito y modelo de aislamiento

Un Orquestador maneja N personalidades; dentro de cada personalidad hay muchos usuarios, cada uno con su propia memoria:

ORQUESTADOR DE IA

```

├── Astrólogo   → Mario1, Mario2, ... Mario∞   (cada uno su memoria)
├── Letrado     → Luis1,  Luis2,  ... Luis∞
├── Psicólogo   → Maria1, Maria2, ... Maria∞
└── Religioso  → Juan1,  Juan2,  ... Juan∞

```

La regla central: la memoria se aísla por el **par (personalidad/app + usuario)**. Lo que conversa Juan1 con el Religioso no debe aparecer en Juan2 (mismo Religioso, otro usuario) ni en Mario1 (otra personalidad).

4. Arquitectura de almacenamiento de memoria

La memoria de cada usuario se materializa en dos entidades: Conversación y Mensaje. La Conversación es la unidad de aislamiento; los Mensajes cuelgan de ella.

4.1 Entidades y campos

Conversación

Campo	Descripción
id	Identificador único de la conversación
app_id	Personalidad/app a la que pertenece (tenant)
usuario_id	Usuario dueño de la conversación
activa	Si es la conversación vigente del par
resumen	Resumen de memoria (opcional, cacheable)
creada	Marca temporal de creación

Mensaje

Campo	Descripción
id	Identificador único del mensaje
conversacion_id	Conversación a la que pertenece (llave foránea)
role	"user" o "assistant"
content	Texto del mensaje
ts	Marca temporal (orden cronológico)

4.2 La llave de aislamiento

La memoria se localiza SIEMPRE por la combinación (**app_id + usuario_id**), que resuelve una Conversación; los Mensajes se traen por conversacion_id:

```
(app_id + usuario_id) → Conversación (activa) → Mensajes
  personalidad+usuario      la llave          la memoria
```

Código real del localizador de conversación (src/orchestrator/memory.js):

```
db.conversaciones.findOne(
  (c) => c.app_id === appId
    && c.usuario_id === usuarioId
    && c.activa
);
```

4.3 Flujo de datos por petición

- 1. Llega la petición con (app_id, usuario_id, pregunta).
- 2. getConversacion(app_id, usuario_id) localiza o crea la conversación del par.
- 3. guardarMensaje() añade el mensaje del usuario a esa conversación.
- 4. historial()/resumir() leen SOLO los mensajes de esa conversacion_id y se inyectan como memoria al prompt.
- 5. La respuesta de la IA se guarda en la misma conversación.

4.4 Implementación actual (en memoria) y ruta a PostgreSQL

Hoy el almacén es en memoria del proceso, con una tabla genérica (estilo OdoO) que expone find/insert/remove. Persiste mientras el servidor vive y se pierde al reiniciar. La llave ya es la correcta, por lo que migrar a PostgreSQL es reemplazar la capa de datos manteniendo la misma interfaz, sin tocar el orquestador ni la memoria.

Esquema previsto en PostgreSQL (con índices que garantizan aislamiento y escala):

```
CREATE TABLE conversacion (  
  id          TEXT PRIMARY KEY,  
  app_id      TEXT NOT NULL,  
  usuario_id  TEXT NOT NULL,  
  activa      BOOLEAN NOT NULL DEFAULT true,  
  resumen     TEXT,  
  creada      TIMESTAMPTZ NOT NULL DEFAULT now()  
);  
CREATE INDEX ix_conv_par ON conversacion (app_id, usuario_id, activa);  
  
CREATE TABLE mensaje (  
  id          TEXT PRIMARY KEY,  
  conversacion_id TEXT NOT NULL REFERENCES conversacion(id) ON DELETE CASCADE,  
  role        TEXT NOT NULL,  
  content     TEXT NOT NULL,  
  ts          TIMESTAMPTZ NOT NULL DEFAULT now()  
);  
CREATE INDEX ix_msg_conv ON mensaje (conversacion_id, ts);
```

Nota: el índice (app_id, usuario_id, activa) hace que localizar la memoria de un par sea $O(\log n)$ y escale a millones de usuarios por personalidad. El borrado en cascada (ON DELETE CASCADE) respalda el derecho LGPD a eliminar.

5. Casos de prueba: cómo funciona cada uno

Cada prueba se ejecuta con el proveedor de IA “mock” (sin costo) a través del orquestador real, de modo que la memoria se guarda y se lee por los mismos caminos que en producción. Las personalidades usadas: aurora = Religioso, sina = Astrólogo.

Prueba A — Aislamiento entre usuarios de la misma personalidad

Cómo funciona el caso: Dos usuarios distintos (Juan1 y Juan2) en la MISMA personalidad (Religioso). Juan1 revela un dato íntimo (un divorcio). Luego se inspecciona la memoria de Juan2.

Qué demuestra del sistema: La memoria de Juan2 NO contiene ningún dato de Juan1. Cada uno tiene una Conversación distinta, localizada por su propio (app_id + usuario_id).

Resultado: ✓ La memoria de Juan2 no contiene el divorcio de Juan1.

Prueba B — Continuidad del mismo usuario

Cómo funciona el caso: Juan1 aporta un dato y, en una interacción posterior (nueva “sesión”), vuelve. Se verifica que la IA recuerde lo anterior.

Qué demuestra del sistema: La memoria de Juan1 persiste entre interacciones mientras el proceso vive; resumir() devuelve el dato previo.

Resultado: ✓ Al volver, la IA recuerda el divorcio de Juan1.

Prueba C — Aislamiento entre personalidades

Cómo funciona el caso: Mario1 aporta un dato en el Astrólogo (sina). Se verifica que ese dato no aparezca en ninguna memoria del Religioso (aurora). Además se comprueba que el mismo usuariold en dos personalidades produce dos Conversaciones distintas.

Qué demuestra del sistema: El dato del Astrólogo nunca aparece en el Religioso. La llave incluye app_id, por lo que un mismo usuariold en dos apps genera memorias separadas.

Resultado: ✓ Dato de Mario (Astrólogo) ausente en el Religioso; conversaciones distintas confirmadas.

Prueba D — Escala (muchos usuarios)

Cómo funciona el caso: Se crean 12 usuarios en la misma personalidad (Religioso), cada uno con un dato secreto único (un “código”). Se verifica que cada memoria contenga SOLO su propio dato.

Qué demuestra del sistema: Los 12 usuarios mantienen memoria propia sin contaminación cruzada.

Resultado: ✓ 12/12 con memoria propia y sin contaminación de otros.

Prueba E — Borrado LGPD (derecho a eliminar)

Cómo funciona el caso: Se invoca borrarMemoriaUsuario(app, usuario) para Juan1 y se verifica que se elimine SOLO su memoria, dejando intactos a los demás usuarios.

Qué demuestra del sistema: Se borran únicamente las conversaciones y mensajes del par (app + usuario) indicado.

Resultado: ✓ Se eliminó solo la memoria de Juan1 (1 conversación, 4 mensajes); los demás intactos.

5.1 Evidencia de ejecución (salida del script)

```
A · ¿Juan2 sabe del divorcio de Juan1? → NO (aislado)
B · Juan1 vuelve → "El usuario antes mencionó: ...divorcio..." → recuerda
C · ¿Dato de Mario (Astrólogo) aparece en Religioso? → NO
  u_juan1 en Religioso → conv con_wps0pmmm
  usuario en Astrólogo → conv con_f37o2jgp (conversaciones distintas)
D · Usuarios con memoria propia y SIN contaminación: 12/12
E · Borrado de u_juan1 → conversaciones:1, mensajes:4; otro usuario intacto
```

6. Respuestas a las preguntas de arquitectura

6.1. ¿Cuál es la llave que separa una memoria de otra?

La entidad Conversación, localizada por la combinación (app_id + usuario_id); los Mensajes cuelgan de conversacion_id. Es por par, no solo por usuario ni solo por personalidad.

6.2. ¿Es persistente o solo dura la sesión? ¿Cómo pasa a PostgreSQL sin reescribir el núcleo?

Hoy persiste mientras el proceso vive y se pierde al reiniciar (almacén en memoria). La migración consiste en reemplazar la capa de datos por un repositorio PostgreSQL manteniendo la interfaz find/insert/remove; el orquestador y la memoria no cambian.

6.3. ¿Hay límite de usuarios por personalidad? ¿Escala a millones?

No hay límite lógico. En memoria es $O(n)$ sobre estructuras de mapa; en PostgreSQL, con el índice (app_id, usuario_id, activa), la búsqueda es $O(\log n)$ y escala a millones por personalidad. El cuello de botella esperado es la base, mitigado con índices, réplicas y caché.

6.4. ¿Cómo se evita técnicamente traer la memoria de otro usuario o personalidad?

Toda consulta de memoria filtra por app_id Y usuario_id juntos; el historial se trae solo por conversacion_id (que ya pertenece a ese par). No existe ningún lookup global por solo-usuario o solo-app, ni estado global de “usuario actual”. Adicionalmente, la validación de usuario exige que el usuario pertenezca a la app.

6.5. ¿Respeto el borrado de datos del usuario (LGPD)?

Sí. La función borrarMemoriaUsuario(app, usuario) elimina solo las conversaciones y mensajes de ese par. En PostgreSQL, el ON DELETE CASCADE garantiza el borrado completo y consistente.

7. Conclusión

El núcleo aísla la memoria por la combinación (personalidad/app + usuario), de forma verificable y reproducible. Las pruebas A–E confirman el aislamiento entre usuarios, la continuidad por usuario, el aislamiento entre personalidades, el comportamiento a escala y el borrado LGPD.

Pendiente único: conectar la persistencia física (PostgreSQL). La llave de aislamiento ya es la correcta, por lo que este paso no requiere rediseñar el núcleo. El pilar del producto —una IA que conoce solo a cada usuario, dentro de cada personalidad, sin mezclarse— queda confirmado.

Reproducibilidad: npm install && node scripts/test-memoria.js